



FPT UNIVERSITY

Course Project Report

Subject: MCP201

Mentor: Tran Ngoc Hoang

Group 4:

1. CE200509 - Le Hieu Dat
2. CS200558 - Duong Van Hoa
3. CE200046 - Nguyen Gia Thuan
4. CE200171 - Nguyen Nhu Van Anh
5. CE200736 - Nguyen Quoc Kiet

– CanTho, July 2026 –

Table of Contents

I. INTRODUCTION

A. Project Information

B. Project Team

II. OVERVIEW

A. Major Features

1. Line Tracking Mode

2. BLUETOOTH Mode

B. System block diagram

III. CODE

IV. CONCLUSION

I. INTRODUCTION

A. Project Information:

1. Project name: **Car Control Using Microcontroller**
2. Group: 4

B. Project Team

Full Name	Role	Email
Nguyen Quoc Kiet	Member	kietnqce200736@gmail.com
Le Hieu Dat	Member	datlhce200509@gmail.com
Nguyen Gia Thuan	Member	thuangce200046@gmail.com
Nguyen Nhu Van Anh	Member	anhnnvce200171@gmail.com
Duong Van Hoa	Leader	hoadvcs200558@gmail.com

II. OVERVIEW

A. Major Features:

1. Line Tracking Mode:

- 3 infrared sensors reading 0/1 values
- Table of cases: line in the middle, shifted left - right, intersection line lost

→ Can illustrate an image of the 3 sensors with different line positions

Left	Center	Right	Error	Meaning
0	1	0	0	Perfectly centered on the line
0	1	1	1	Line drifting right - slight correction
0	0	1	2	Line far right - stronger correction
1	1	0	-1	Line drifting left - slight correction
1	0	0	-2	Line far left - stronger correction
1	1	1	0	Crossroad / intersection - go straight
0	0	0	± 4	Line lost - use lastError sign for a hard recovery pivot

- *The PD Control Algorithm:*

- + Proportional: reacts to how far off-line the car currently is: **$P = K_p \times \text{error}$**
- + Derivative: reacts to how fast the error is changing - this brakes overshoot:

$$D = K_d \times (\text{error} - \text{lastError})$$

- + Sum: **correction = P + D**

2. BLUETOOTH Mode:

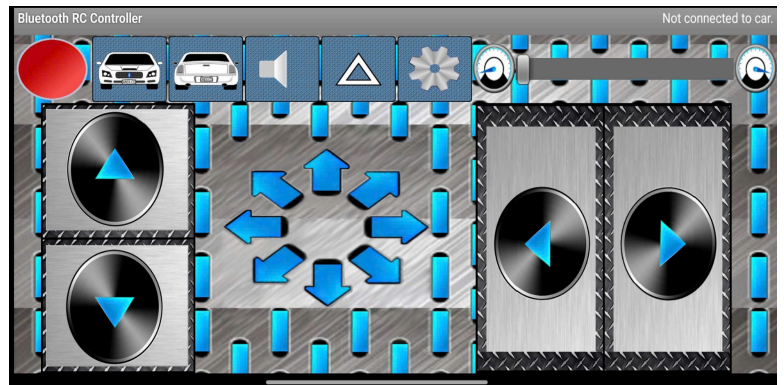
- Use the UART protocol to communicate
- Explain the character commands: **F/B/L/R** (forward/back/left/right), **G/I/H/J** (diagonal), **S** (stop)
- Character **X/W** turns on line-following mode, **x/w** turns it off (uppercase/lowercase distinguish on/off)
- Manual Mode: Bluetooth Commands: Each byte from the phone app maps directly to a differential motor-speed pair:
 - + **Forward:** setMotorSpeeds(MAX, MAX)
 - + **Backward:** setMotorSpeeds(-MAX, -MAX)
 - + **Fwd-Left:** setMotorSpeeds(0.5*MAX, MAX)
 - + **Fwd-Right:** setMotorSpeeds(MAX, 0.5*MAX)
 - + **Spin Left:** setMotorSpeeds(-MAX, MAX)
 - + **Spin Right:** setMotorSpeeds(MAX, -MAX)
 - + **Back - Left:** setMotorSpeeds(-0.5*MAX, -MAX)
 - + **Back - Right:** setMotorSpeeds(-0.5*MAX, -MAX)
 - + **Stop:** stopCar()



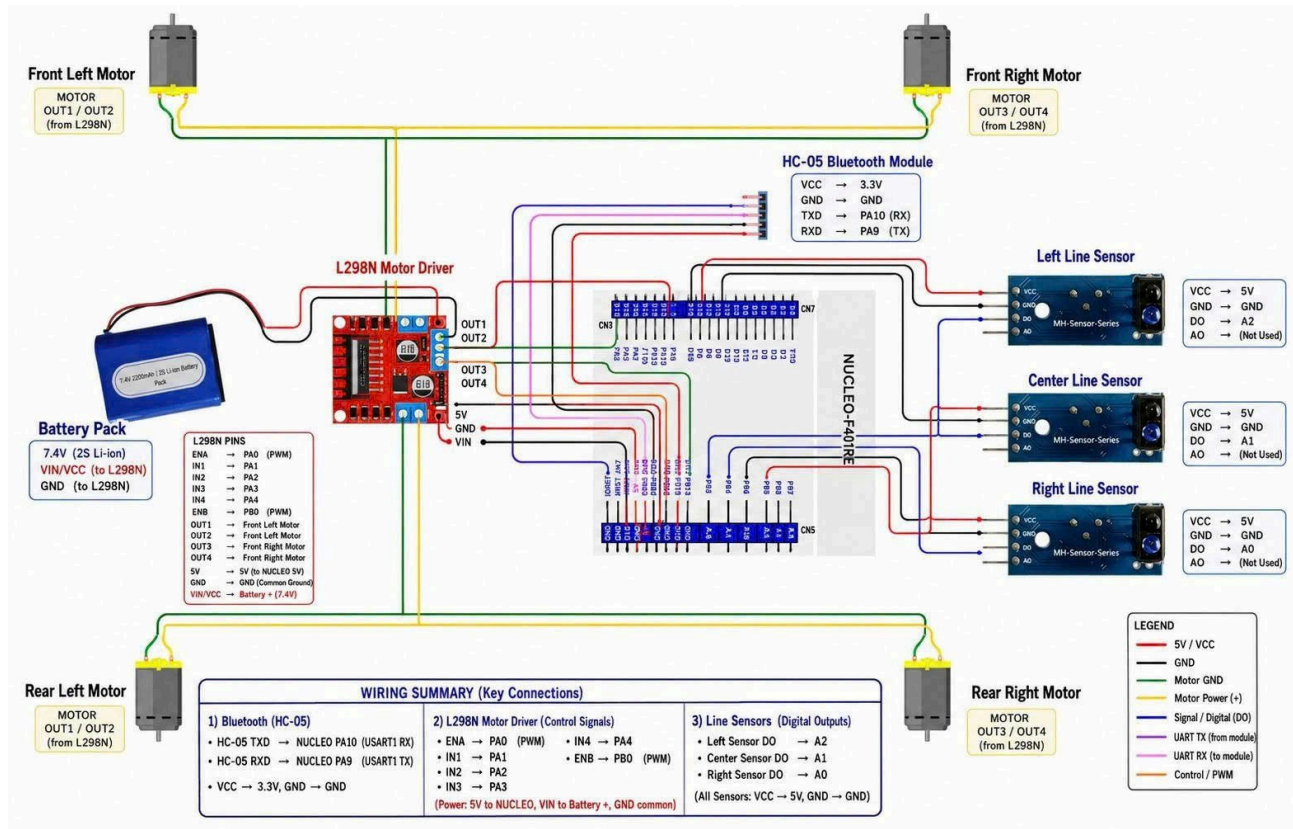
Bluetooth RC Car APK dành cho Android

Miễn phí ★ 4.5 (65 ⭐) ↓ 8.2K

✓ Chương trình đáng tin cậy V. 2.1



B. System Block Diagram:



- **Controller:** NUCLEO-F411RE (mbed)
 - **Power block:** L298N + 2 DC motors
 - **Sensor block:** 3 line-following sensors (MH-Sensor)
 - **Communication block:** Bluetooth module (HC-05 via UART)
- (App ↔ Bluetooth ↔ MCU ↔ L298N ↔ Motor, and Sensor → MCU)

III. Code:

```
#include "mbed.h"
#include <chrono>
#include <cmath>

using namespace std::chrono;

BufferedSerial bt(D8, D2, 9600);

// L298N Motor Driver - Speed Control (PWM)
PwmOut ENA(D11);
PwmOut ENB(D10);

BusOut motor_pins(D3, D4, D5, D6);

// MH-Sensor Series Line Tracking Pins
DigitalIn sensorLeft(D7);
DigitalIn sensorCenter(D9);
DigitalIn sensorRight(D12);

bool isLineTrackingMode = false;

const float MAX_SPEED      = 0.98f;
const float LINE_MAX_SPEED = 0.65f;

float Kp = 0.15f;
float Kd = 0.70f;

int lastError = 0;

Timer connectionTimer;

void stopCar() {
    ENA.write(0.0f);
    ENB.write(0.0f);
}
```

```

    motor_pins.write(0b0000);
}

void setMotorSpeeds(float leftSpeed, float rightSpeed, float
speedLimit = MAX_SPEED) {
    bool leftFwd = (leftSpeed >= 0);
    bool rightFwd = (rightSpeed >= 0);

    leftSpeed = fabs(leftSpeed);
    rightSpeed = fabs(rightSpeed);

    if (leftSpeed > speedLimit) leftSpeed = speedLimit;
    if (rightSpeed > speedLimit) rightSpeed = speedLimit;

    uint8_t dirBits = 0;
    if (leftFwd) dirBits |= 0b0001;
    else dirBits |= 0b0010;
    if (rightFwd) dirBits |= 0b0100;
    else dirBits |= 0b1000;

    motor_pins.write(dirBits);
    ENA.write(leftSpeed);
    ENB.write(rightSpeed);
}

void lineTrackingLogic() {
    int sl = sensorLeft.read();
    int sc = sensorCenter.read();
    int sr = sensorRight.read();

    int error = 0;
    // 1. Calculate Error based on sensor states
    if (sl == 0 && sc == 1 && sr == 0) {
        error = 0;
    }
    else if (sl == 0 && sc == 1 && sr == 1) {
        error = 1;
    }
    else if (sl == 0 && sc == 0 && sr == 1) {

```

```

        error = 2;
    }
    else if (sl == 1 && sc == 1 && sr == 0) {
        error = -1;
    }
    else if (sl == 1 && sc == 0 && sr == 0) {
        error = -2;
    }
    else if (sl == 1 && sc == 1 && sr == 1) {
        error = 0;
    }
    else if (sl == 0 && sc == 0 && sr == 0) {
        if (lastError < 0) {
            error = -4;
        } else if (lastError > 0) {
            error = 4;
        } else {
            error = 0;
        }
    }
}

float P = Kp * error;
float D = Kd * (error - lastError);
float correction = P + D;

lastError = error;

float leftMotorSpeed = LINE_MAX_SPEED + correction;
float rightMotorSpeed = LINE_MAX_SPEED - correction;

setMotorSpeeds(leftMotorSpeed, rightMotorSpeed, LINE_MAX_SPEED);
}

int main() {

    ENA.period_us(50);
    ENB.period_us(50);

    // Start the failsafe timer

```

```

connectionTimer.start();

stopCar();
char rx_char;

while (true) {
    if (bt.readable()) {
        ssize_t n = bt.read(&rx_char, 1);

        if (n > 0) {
            connectionTimer.reset();

            if (rx_char == 'X' || rx_char == 'W') {
                isLineTrackingMode = true;
                lastError = 0;
                stopCar();
            }
            else if (rx_char == 'x' || rx_char == 'w') {
                isLineTrackingMode = false;
                stopCar();
            }

            if (!isLineTrackingMode) {
                switch(rx_char) {
                    case 'F': setMotorSpeeds(MAX_SPEED,
MAX_SPEED); break;
                    case 'B': setMotorSpeeds(-MAX_SPEED,
-MAX_SPEED); break;
                    case 'L': setMotorSpeeds(-MAX_SPEED,
MAX_SPEED); break;
                    case 'R': setMotorSpeeds(MAX_SPEED,
-MAX_SPEED); break;

                    case 'G': setMotorSpeeds(MAX_SPEED * 0.5f,
MAX_SPEED); break; // Forward-Left
                    case 'I': setMotorSpeeds(MAX_SPEED, MAX_SPEED
* 0.5f); break; // Forward-Right
                    case 'J': setMotorSpeeds(-MAX_SPEED * 0.5f,
-MAX_SPEED); break; // Back-Left

```

```

        case 'H': setMotorSpeeds(-MAX_SPEED,
-MAX_SPEED * 0.5f); break; // Back-Right

        case 'S': stopCar(); break;
    }
}

// If no Bluetooth data is received for more than 500ms in
manual mode, stop the car
if (!isLineTrackingMode) {
    auto elapsedTime =
duration_cast<milliseconds>(connectionTimer.elapsed_time()).count();
    if (elapsedTime > 500) {
        stopCar();
    }
}

// 4. Execute Line Tracking continuously if active
if (isLineTrackingMode) {
    lineTrackingLogic();
}

    ThisThread::sleep_for(2ms);
}
}

```

IV. Conclusion:

Within the scope of the "Car Control Using Microcontroller" project for the MCP201 course, our group successfully developed an intelligent control system based on the NUCLEO-F411RE microcontroller running on Mbed OS. The system features a seamless integration of hardware and software, utilizing the UART protocol for Bluetooth communication and Pulse Width Modulation (PWM) signals with an L298N H-bridge for precise DC motor control. Furthermore, by implementing a PD control algorithm to process real-time infrared sensor data, the vehicle achieves stable and accurate autonomous line-following performance, successfully fulfilling the objectives established by our team.